

ECMAScript REGEXP: LINEAR IMPLEMENTATIONS AND FORMAL METHODS

AURÈLE BARRIÈRE (CNRS) · CLÉMENT PIT-CLAUDEL (EPFL)

Our specialty: formal methods for programming languages

Semantics, compilation and execution techniques, formal verification (Rocq/Coq proof assistant).

In the last 3 years: formal methods for modern regexes.

Our specialty: formal methods for programming languages

Semantics, compilation and execution techniques, formal verification (Rocq/Coq proof assistant).

In the last 3 years: formal methods for modern regexes.

What this talk is about:

Linear-Time Regex Matching

- Linear-time matching solves concrete issues.
- ECMAScript is surprisingly adapted for linear-time matching.
Only language for which we can support all lookarounds in linear-time.
- When adding features, matching complexity should be considered.

Our specialty: formal methods for programming languages

Semantics, compilation and execution techniques, formal verification (Rocq/Coq proof assistant).

In the last 3 years: formal methods for modern regexes.

What this talk is about:

Linear-Time Regex Matching

- Linear-time matching solves concrete issues.
- ECMAScript is surprisingly adapted for linear-time matching.
Only language for which we can support all lookarounds in linear-time.
- When adding features, matching complexity should be considered.

Formal Methods

- Formal methods can bring a lot to modern regexes.
- ECMAScript regexes have an elegant semantics, suitable for formal methods.
- We are verifying properties and developing verified tools/engines.

Most engines have exponential complexity

```
"a".repeat(15).match(/(a*)*b/): 1 ms
```

Most engines have exponential complexity

"a".repeat(15).match(/(a*)*b/): 1 ms

"a".repeat(35).match(/(a*)*b/): 12 minutes

Most engines have exponential complexity

"a".repeat(15).match(/(a*)*b/): 1 ms

"a".repeat(35).match(/(a*)*b/): 12 minutes

"a".repeat(100).match(/(a*)*b/) would take 10^{14} years!

Most engines have exponential complexity

"a".repeat(15).match(/(a*)*b/): 1 ms

"a".repeat(35).match(/(a*)*b/): 12 minutes

"a".repeat(100).match(/(a*)*b/) would take 10^{14} years!

Most engines have exponential complexity

`"a".repeat(15).match(/(a*)*b/)` : 1 ms

`"a".repeat(35).match(/(a*)*b/)` : 12 minutes

`"a".repeat(100).match(/(a*)*b/)` would take 10^{14} years!

Regex performance matters

ReDoS Vulnerability: Regular Expression Denial Of Service

- 12% of JavaScript-based web servers were found vulnerable **[USENIX'18]**.
- Major services (Cloudflare, StackOverflow) brought down due to regex performance issues.
- It's easy to run into these issues: `/. *a . *b /` has quadratic complexity.
- Also an issue when using regexes as a target language.
E.g. Glob patterns to regexes: **[CVE-2026-27903; CVE-2026-27904; CVE-2026-26996]**

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

```
"abcd".match(/ab*|abc/) = "ab".
```

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a
Disjunction	r1 r2
Quantifiers	r*

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*			

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.	
Disjunction	r1 r2				
Quantifiers	r*	r+	r*?	r{n,m}	—————> Matches r from n to m times.

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*	r+	r*?	r{n,m}
Capture Groups	(r)			

→ Returns the substring last matched by subexpressions in parentheses.

`"abcd".match(/a(b*)|abc/) = ["ab", "b"]`

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*	r+	r*?	r{n,m}
Capture Groups	(r)			
Anchors	^	\$		

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab"`.

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*	r+	r*?	r{n,m}
Capture Groups	(r)			
Anchors	^	\$		
Lookarounds	(?=r)	(?!r)		

→ Adds a condition in a regex:

`/a(=b)/` matches "a" only if followed by "b".

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*	r+	r*?	r{n,m}
Capture Groups	(r)			
Anchors	^	\$		
Lookarounds	(?=r)	(?!r)	(?<=r)	(?<!r)

Matches the string backwards:

`"TC39".match(/(?<=TC)\d+/) = "39".`

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*	r+	r*?	r{n,m}
Capture Groups	(r)			
Anchors	^	\$		
Lookarounds	(?=r)	(?!r)	(?<=r)	(?<!=r)
Backreferences	\1	\k<name>		

Backtracking Semantics

Backtracking Semantics: Return the first match found by a backtracking algorithm.

`"abcd".match(/ab*|abc/) = "ab".`

Features (non-exhaustive)

Characters	a	[a-z]	\d	.
Disjunction	r1 r2			
Quantifiers	r*	r+	r*?	r{n,m}
Capture Groups	(r)			
Anchors	^	\$		
Lookarounds	(?=r)	(?!r)	(?<=r)	(?<!r)
Backreferences	\1	\k<name>		

Some languages (Rust,Go) omit features (backreferences, lookarounds) and guarantee linear-time. Others (.NET) provide both an exponential-time engine for all features, and a linear one for a subset.

The V8 Experimental Engine

Behind V8 flag `--enable-experimental-regexp-engine` and regex flag `/l`.

```
"a".repeat(100).match(/(a*)*b/l) : 0.01 ms.
```

Some unsupported features: `u` and `v` flags, duplicate named groups...

Recent proposal (Feb 2026 – Stage 0): Regular Expression Linear-Time Flag.

The V8 Experimental Engine

Behind V8 flag `--enable-experimental-regexp-engine` and regex flag `/l`.

`"a".repeat(100).match(/(a*)*b/l) : 0.01 ms.`

Some unsupported features: `u` and `v` flags, duplicate named groups...

Recent proposal (Feb 2026 – Stage 0): Regular Expression Linear-Time Flag.

Libraries for linear matching

Missing features. Common features may have different semantics.

- `node-re2`: Node.js bindings for the RE2 linear engine.
- `regolith`: TypeScript/JavaScript server-side lib using the Rust linear engine.
- `re2js`: RE2 ported to JavaScript.
Uses our linear-time algorithm for captureless lookbehinds.
- `@iter-tools/regex` and `@bablr/regex-vm`: streaming regex implementation.

The V8 Experimental Engine

Behind V8 flag `--enable-experimental-regexp-engine` and regex flag `/l`.

`"a".repeat(100).match(/(a*)*b/l) : 0.01 ms.`

Some unsupported features: `u` and `v` flags, duplicate named groups...

Recent proposal (Feb 2026 – Stage 0): Regular Expression Linear-Time Flag.

Libraries for linear matching

Missing features. Common features may have different semantics.

- `node-re2`: Node.js bindings for the RE2 linear engine.
- `regolith`: TypeScript/JavaScript server-side lib using the Rust linear engine.
- `re2js`: RE2 ported to JavaScript.
Uses our linear-time algorithm for captureless lookbehinds.
- `@iter-tools/regex` and `@bablr/regex-vm`: streaming regex implementation.

There is interest for linear-time matching in JavaScript

MATCHING ECMAScript REGEXES IN LINEAR TIME

Quantifiers (`*`, `+`, `?`, `*?`, `+?`, `??`, ...) are different than in any other regex language!
These subtle differences have an impact on matching complexity.

22.2.2.3.1 RepeatMatcher (*m*, *min*, *max*, *greedy*, *x*, *c*, *parenIndex*, *parenCount*)

The abstract operation RepeatMatcher takes arguments *m* (a [Matcher](#)), *min* (a non-negative [integer](#)), *max* (a non-negative [integer](#) or $+\infty$), *greedy* (a Boolean), *x* (a [MatchState](#)), *c* (a [MatcherContinuation](#)), *parenIndex* (a non-negative [integer](#)), and *parenCount* (a non-negative [integer](#)) and returns either a [MatchState](#) or FAILURE. It performs the following steps when called:

1. If *max* = 0, return *c*(*x*).
2. Let *d* be a new [MatcherContinuation](#) with parameters (*y*) that captures *m*, *min*, *max*, *greedy*, *x*, *c*, *parenIndex*, and *parenCount* and performs the following steps when called:
 - a. **Assert**: *y* is a [MatchState](#).
 - b. If *min* = 0 and *y*.[[EndIndex]] = *x*.[[EndIndex]], return FAILURE.
 - c. If *min* = 0, let *min2* be 0; otherwise let *min2* be *min* - 1.
 - d. If *max* = $+\infty$, let *max2* be $+\infty$; otherwise let *max2* be *max* - 1.
 - e. Return [RepeatMatcher](#)(*m*, *min2*, *max2*, *greedy*, *y*, *c*, *parenIndex*, *parenCount*).
3. Let *cap* be a copy of *x*.[[Captures]].
4. For each [integer](#) *k* in the [inclusive interval](#) from *parenIndex* + 1 to *parenIndex* + *parenCount*, set *cap*[*k*] to **undefined**.
5. Let *input* be *x*.[[Input]].
6. Let *e* be *x*.[[EndIndex]].
7. Let *xr* be the [MatchState](#) { [[Input]]: *input*, [[EndIndex]]: *e*, [[Captures]]: *cap* }.
8. If *min* ≠ 0, return *m*(*xr*, *d*).
9. If *greedy* is **false**, then
 - a. Let *z* be *c*(*x*).
 - b. If *z* is not FAILURE, return *z*.
 - c. Return *m*(*xr*, *d*).
10. Let *z* be *m*(*xr*, *d*).
11. If *z* is not FAILURE, return *z*.
12. Return *c*(*x*).

22.2.2.3.1 RepeatMatcher (*m*, *min*, *max*, *greedy*, *x*, *c*, *parenIndex*, *parenCount*)

The abstract operation RepeatMatcher takes arguments *m* (a [Matcher](#)), *min* (a non-negative [integer](#)), *max* (a non-negative [integer](#) or $+\infty$), *greedy* (a Boolean), *x* (a [MatchState](#)), *c* (a [MatcherContinuation](#)), *parenIndex* (a non-negative [integer](#)), and *parenCount* (a non-negative [integer](#)) and returns either a [MatchState](#) or FAILURE. It performs the following steps when called:

1. If *max* = 0, return *c*(*x*).
2. Let *d* be a new [MatcherContinuation](#) with parameters (*y*) that captures *m*, *min*, *max*, *greedy*, *x*, *c*, *parenIndex*, and *parenCount* and performs the following steps when called:
 - a. **Assert**: *y* is a [MatchState](#).
 - b. If *min* = 0 and *y*.[[EndIndex]] = *x*.[[EndIndex]], return FAILURE.
 - c. If *min* = 0, let *min2* be 0; otherwise let *min2* be *min* - 1.
 - d. If *max* = $+\infty$, let *max2* be $+\infty$; otherwise let *max2* be *max* - 1.
 - e. Return [RepeatMatcher](#)(*m*, *min2*, *max2*, *greedy*, *y*, *c*, *parenIndex*, *parenCount*).
3. Let *cap* be a copy of *x*.[[Captures]].
4. For each [integer](#) *k* in the inclusive interval from *parenIndex* + 1 to *parenIndex* + *parenCount*, set *cap*[*k*] to undefined.
5. Let *Input* be *x*.[[Input]].
6. Let *e* be *x*.[[EndIndex]].
7. Let *xr* be the [MatchState](#) { [[Input]]: *Input*, [[EndIndex]]: *e*, [[Captures]]: *cap* }.
8. If *min* ≠ 0, return *m*(*xr*, *d*).
9. If *greedy* is **false**, then
 - a. Let *z* be *c*(*x*).
 - b. If *z* is not FAILURE, return *z*.
 - c. Return *m*(*xr*, *d*).
10. Let *z* be *m*(*xr*, *d*).
11. If *z* is not FAILURE, return *z*.
12. Return *c*(*x*).

Invalid Iterations

Optional (min=0) iterations that do not advance in the string are invalid.

Capture Reset

At each iteration, reset the values of capture groups inside the quantifier.

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

ECMAScript: Empty iterations are invalid.

$((a \mid \epsilon) (\epsilon \mid b))^*$ on "ab"
 $\xrightarrow{a}$

ECMAScript: Empty iterations are invalid.

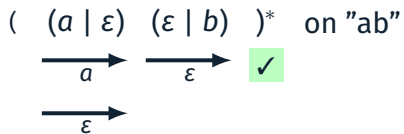
$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"
 $\xrightarrow{a} \xrightarrow{\varepsilon}$

ECMAScript: Empty iterations are invalid.

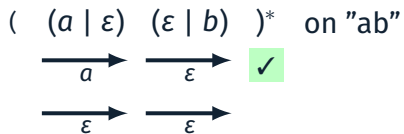
$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

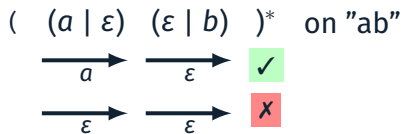
ECMAScript: Empty iterations are invalid.



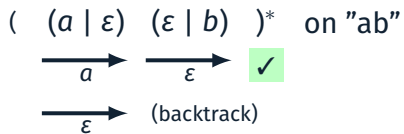
ECMAScript: Empty iterations are invalid.



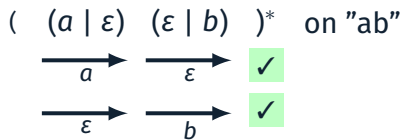
ECMAScript: Empty iterations are invalid.



ECMAScript: Empty iterations are invalid.



ECMAScript: Empty iterations are invalid.



ECMAScript: Empty iterations are invalid.

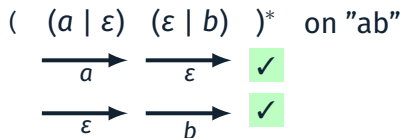
$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{b} \checkmark$

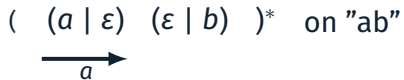
Result: 2 iterations, match "ab".

ECMAScript: Empty iterations are invalid.



Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.



ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{b} \checkmark$

Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon}$

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{b} \checkmark$

Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{b} \checkmark$

Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon}$

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon}$ ✓

$\xrightarrow{\varepsilon} \xrightarrow{b}$ ✓

Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon}$ ✓

$\xrightarrow{\varepsilon} \xrightarrow{\varepsilon}$

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{b} \checkmark$

Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{\varepsilon} \checkmark$ (final)

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{b} \checkmark$

Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

$\xrightarrow{a} \xrightarrow{\varepsilon} \checkmark$

$\xrightarrow{\varepsilon} \xrightarrow{\varepsilon} \checkmark$ (final)

Result: 2 iterations, match "a".

ECMAScript: Empty iterations are invalid.

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"



Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

Rust/Go/RE2: ε -loops are forbidden.

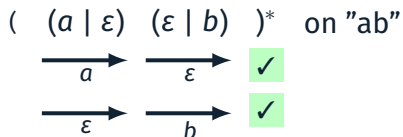
$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"



Result: 2 iterations, match "a".

$((a \mid \varepsilon) (\varepsilon \mid b))^*$ on "ab"

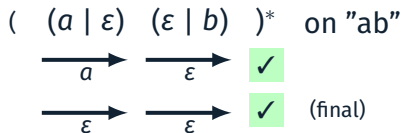
ECMAScript: Empty iterations are invalid.



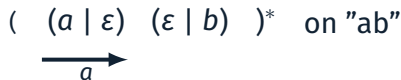
Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

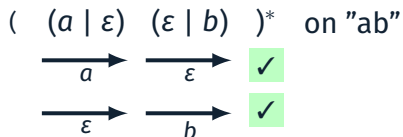
Rust/Go/RE2: ϵ -loops are forbidden.



Result: 2 iterations, match "a".



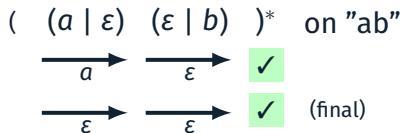
ECMAScript: Empty iterations are invalid.



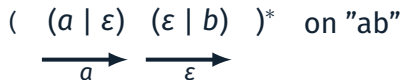
Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

Rust/Go/RE2: ε -loops are forbidden.



Result: 2 iterations, match "a".



ECMAScript: Empty iterations are invalid.

$((a \mid \epsilon) (\epsilon \mid b))^*$ on "ab"



Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

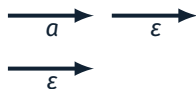
Rust/Go/RE2: ϵ -loops are forbidden.

$((a \mid \epsilon) (\epsilon \mid b))^*$ on "ab"

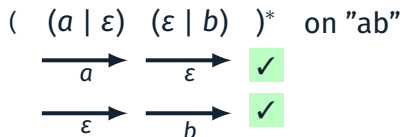


Result: 2 iterations, match "a".

$((a \mid \epsilon) (\epsilon \mid b))^*$ on "ab"



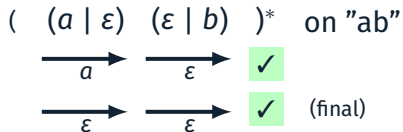
ECMAScript: Empty iterations are invalid.



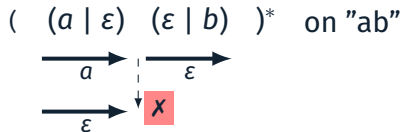
Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

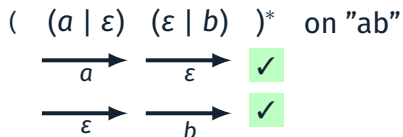
Rust/Go/RE2: ϵ -loops are forbidden.



Result: 2 iterations, match "a".



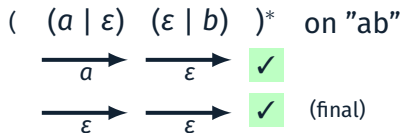
ECMAScript: Empty iterations are invalid.



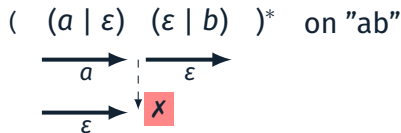
Result: 2 iterations, match "ab".

PCRE/Python/Java/.NET: Empty iterations are final.

Rust/Go/RE2: ϵ -loops are forbidden.



Result: 2 iterations, match "a".



Result: 1 iteration, match "a".

We needed new algorithms for the star!

Standard linear-time algorithms only support Rust/Go/RE2 semantics, with complexity $O(|r| \times |s|)$.

We needed new algorithms for the star!

Standard linear-time algorithms only support Rust/Go/RE2 semantics, with complexity $O(|r| \times |s|)$.
We designed a variant of these algorithms for ECMAScript semantics, with complexity $O(|r| \times |s|)$.
We fixed V8 Experimental.

We needed new algorithms for the star!

Standard linear-time algorithms only support Rust/Go/RE2 semantics, with complexity $O(|r| \times |s|)$.

We designed a variant of these algorithms for EMCAScript semantics, with complexity $O(|r| \times |s|)$.

We fixed V8 Experimental.

The best we found for PCRE/Python/Java/.NET has $O(|r|^2 \times |s|)$ complexity.

We needed new algorithms for the star!

Standard linear-time algorithms only support Rust/Go/RE2 semantics, with complexity $O(|r| \times |s|)$.

We designed a variant of these algorithms for ECMAScript semantics, with complexity $O(|r| \times |s|)$.

We fixed V8 Experimental.

The best we found for PCRE/Python/Java/.NET has $O(|r|^2 \times |s|)$ complexity.

Regex-size complexity

Linear-time engines are typically not regex-size linear for counted quantifiers:

$r\{n,m\}$ is unrolled. Complexity $O(2^{|r|} \times |s|)$.

For the plus ($r+$, $r+?$), other languages can avoid this unfolding. But in ECMAScript, we found linear-time algorithms only when r is non-nullable or the plus is greedy.

Can we deal with the lazy nullable plus in regex-size linear time?

(appears in 0.003% of regexes)

Capture Reset

At each quantifier iteration, reset the value of groups inside.

```
"ab".match(/(?:a|b)*/) = ["ab", undefined]
```

Consequence: Only the last iteration of a quantifier can define groups.

Consequence: Each capture group inside a lookahead can only be defined by the last time the lookahead was used.

Capture Reset

At each quantifier iteration, reset the value of groups inside.

```
"ab".match(/(?:a|b)*/) = ["ab", undefined]
```

Consequence: Only the last iteration of a quantifier can define groups.

Consequence: Each capture group inside a lookahead can only be defined by the last time the lookahead was used.

Our three-steps linear algorithm

- Find all positions where lookarounds hold.
- Match the main expression.
- Reconstruct groups in lookarounds:
Match each lookahead **once** from where they were last used.

This algorithm has now been implemented in V8 Experimental.

Current state of linearity

- Without backreferences, everything can be matched in string-size linear time.
- Without counted quantifiers and the lazy nullable plus, everything can be matched in regex-size linear time.

Current state of linearity

- Without backreferences, everything can be matched in string-size linear time.
- Without counted quantifiers and the lazy nullable plus, everything can be matched in regex-size linear time.

Current proposals that should not affect linearity

- Stage 3: Legacy Features
- Stage 2: Buffer Boundaries
- Stage 1: Extended mode
- Stage 1: \R escape

Current state of linearity

- Without backreferences, everything can be matched in string-size linear time.
- Without counted quantifiers and the lazy nullable plus, everything can be matched in regex-size linear time.

Current proposals that should not affect linearity

- Stage 3: Legacy Features
- Stage 2: Buffer Boundaries
- Stage 1: Extended mode
- Stage 1: \R escape

Can we handle this in linear time?

- Stage 1: Atomic operators

Current state of linearity

- Without backreferences, everything can be matched in string-size linear time.
- Without counted quantifiers and the lazy nullable plus, everything can be matched in regex-size linear time.

Current proposals that should not affect linearity

- Stage 3: Legacy Features
- Stage 2: Buffer Boundaries
- Stage 1: Extended mode
- Stage 1: \R escape

Can we handle this in linear time?

- Stage 1: Atomic operators

Recent work [ESOP'24] presents a linear-time algorithm for atomic groups, but it does not work if the inner regex is nullable!

FORMAL VERIFICATION FOR ECMAScript REGEXES

22.2.2.4.1 `IsWordChar ( rer, Input, e )`

The abstract operation `IsWordChar` takes arguments `rer` (a `RegExp` Record), `Input` (a `List` of characters), and `e` (an `integer`) and returns a `Boolean`. It performs the following steps when called:

1. Let `InputLength` be the number of elements in `Input`.
2. If `e = -1` or `e = InputLength`, return `false`.
3. Let `c` be the character `Input[ e ]`.
4. If `WordCharacters(rer)` contains `c`, return `true`.
5. Return `false`.

```
(** >>
```

```
  22.2.2.4.1 IsWordChar ( rer, Input, e )
```

The abstract operation IsWordChar takes arguments rer (a RegExp Record), Input (a List of characters), and e (an integer) and returns a Boolean. It performs the following steps when called:

```
<<*)
```

```
(*>> 1. Let InputLength be the number of elements in Input. <<*)
```

```
(*>> 2. If e = -1 or e = InputLength, return false. <<*)
```

```
(*>> 3. Let c be the character Input[ e ]. <<*)
```

```
(*>> 4. If WordCharacters(rer) contains c, return true. <<*)
```

```
(*>> 5. Return false. <<*)
```

```
(** >>
```

```
  22.2.2.4.1 IsWordChar ( rer, Input, e )
```

The abstract operation IsWordChar takes arguments rer (a RegExp Record), Input (a List of characters), and e (an integer) and returns a Boolean. It performs the following steps when called:

```
<<*)
Definition isWordChar (rer: RegExpRecord) (Input: list Character) (e: integer): Result bool :=
  (*>> 1. Let InputLength be the number of elements in Input. <<*)
  let InputLength := List.length Input in
  (*>> 2. If e = -1 or e = InputLength, return false. <<*)
  if (e =? -1)%Z || (e =? InputLength)%Z then false
  else
  (*>> 3. Let c be the character Input[ e ]. <<*)
  let! c =<< Input[e] in
  (*>> 4. If WordCharacters(rer) contains c, return true. <<*)
  let! wc =<< wordCharacters rer in
  if CharSet.contains wc c then true
  else
  (*>> 5. Return false. <<*)
  false.
```


An executable mechanization

We ran Test262 with this code.
498 relevant regex tests.

An executable mechanization

We ran Test262 with this code.

498 relevant regex tests. 495 passed, 3 timeouts.

An executable mechanization

We ran Test262 with this code.

498 relevant regex tests. 495 passed, 3 timeouts.

Limitations

- ECMAScript 14th edition (2023): no v flag, no duplicate named groups...
- Parameterized by some unicode functions.
- We have not translated regex parsing.

An executable mechanization

We ran Test262 with this code.

498 relevant regex tests. 495 passed, 3 timeouts.

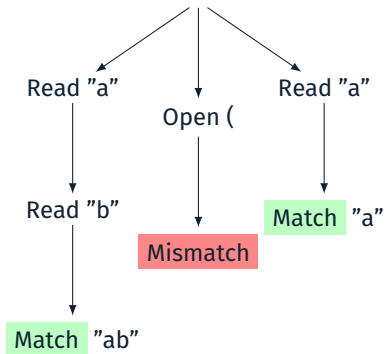
Limitations

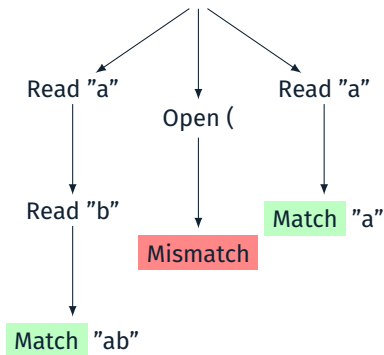
- ECMAScript 14th edition (2023): no v flag, no duplicate named groups...
- Parameterized by some unicode functions.
- We have not translated regex parsing.
- Not ideal for formal verification.

The ECMAScript style is a great intuitive specification for backtracking semantics.
But what would be the ideal semantics for formal verification?

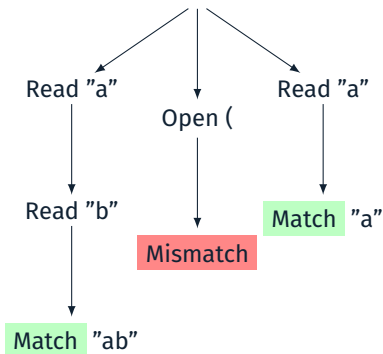
$$\begin{array}{c}
\frac{}{([], i, gm, d) \Downarrow \text{Match}} \text{MATCH} \\
\\
\frac{i_{\text{check}} <_d i \quad (l, i, gm, d) \Downarrow t}{(\text{Acheck } i_{\text{check}} :: l, i, gm, d) \Downarrow \text{Progress } t} \text{CHECK} \quad \frac{\neg(i_{\text{check}} <_d i)}{(\text{Acheck } i_{\text{check}} :: l, i, gm, d) \Downarrow \text{Mismatch}} \text{CHECKFAIL} \\
\\
\frac{\text{advance}(cd, i, d) = \text{Some}(c, i') \quad (l, i', gm, d) \Downarrow t}{(cd :: l, i, gm, d) \Downarrow \text{Read } c \ t} \text{READ} \quad \frac{\text{advance}(cd, i, d) = \text{None}}{(cd :: l, i, gm, d) \Downarrow \text{Mismatch}} \text{READFAIL} \\
\\
\frac{(r_1 :: l, i, gm, d) \Downarrow t_1 \quad (r_2 :: l, i, gm, d) \Downarrow t_2}{((r_1 | r_2) :: l, i, gm, d) \Downarrow \text{Choice } t_1 \ t_2} \text{DISJ} \\
\\
\frac{(r_1 :: r_2 :: l, i, gm, \rightarrow) \Downarrow t}{(r_1 \ r_2 :: l, i, gm, \rightarrow) \Downarrow t} \text{SEQFORWARD} \quad \frac{(r_2 :: r_1 :: l, i, gm, \leftarrow) \Downarrow t}{(r_1 \ r_2 :: l, i, gm, \leftarrow) \Downarrow t} \text{SEQBACKWARD} \\
\\
\frac{(l, i, gm, d) \Downarrow t}{(\epsilon :: l, i, gm, d) \Downarrow t} \text{EPSILON} \quad \frac{(r :: \text{Aclose } g :: l, i, \text{GM}_{\text{open}}(gm, g, \text{idx}(i)), d) \Downarrow t}{((g \ r) :: l, i, gm, d) \Downarrow \text{Open } g \ t} \text{GROUP} \\
\\
\frac{\text{check_anchor}(a, i) = \top \quad (l, i, gm, d) \Downarrow t}{(a :: l, i, gm, d) \Downarrow \text{AnchorPass } a \ t} \text{ANCHOR} \quad \frac{\text{check_anchor}(a, i) = \perp}{(a :: l, i, gm, d) \Downarrow \text{Mismatch}} \text{ANCHORFAIL} \\
\\
\frac{\text{advance_backref}(gm, g, i, d) = \text{Some}(s, i') \quad (l, i', gm, d) \Downarrow t}{(\backslash g :: l, i, gm, d) \Downarrow \text{BackrefPass } s \ t} \text{BACKREF} \quad \frac{\text{advance_backref}(gm, g, i, d) = \text{None}}{(\backslash g :: l, i, gm, d) \Downarrow \text{Mismatch}} \text{BACKREFFAIL} \\
\\
\frac{(r :: r\{\min, \Delta, p\} :: l, i, \text{GM}_{\text{reset}}(gm, \mathcal{G}(r)), d) \Downarrow t}{(r\{\min + 1, \Delta, p\} :: l, i, gm, d) \Downarrow \text{Reset } \mathcal{G}(r) \ t} \text{FORCED} \quad \frac{(l, i, gm, d) \Downarrow t}{(r\{0, 0, p\} :: l, i, gm, d) \Downarrow t} \text{DONE} \\
\\
\frac{(l, i, gm, d) \Downarrow t_{\text{skip}} \quad (r :: \text{Acheck } i :: r\{0, \Delta, \top\} :: l, i, \text{GM}_{\text{reset}}(gm, \mathcal{G}(r)), d) \Downarrow t_{\text{iter}}}{(r\{0, \Delta + 1, \top\} :: l, i, gm, d) \Downarrow \text{Choice } (\text{Reset } \mathcal{G}(r) \ t_{\text{iter}}) \ t_{\text{skip}}} \text{GREEDY} \\
\\
\frac{(l, i, gm, d) \Downarrow t_{\text{skip}} \quad (r :: \text{Acheck } i :: r\{0, \Delta, \perp\} :: l, i, \text{GM}_{\text{reset}}(gm, \mathcal{G}(r)), d) \Downarrow t_{\text{iter}}}{(r\{0, \Delta + 1, \perp\} :: l, i, gm, d) \Downarrow \text{Choice } t_{\text{skip}} \ (\text{Reset } \mathcal{G}(r) \ t_{\text{iter}})} \text{LAZY} \\
\\
\frac{\text{dir}(lk) = d' \quad \text{lk_result}(lk, t_{\text{look}}, gm, \text{idx}(i)) = \text{Some } gm' \quad ([r], i, gm, d') \Downarrow t_{\text{look}} \quad (l, i, gm', d) \Downarrow t}{((?lk \ r) :: l, i, gm, d) \Downarrow \text{LK } lk \ t_{\text{look}} \ t} \text{LOOKAROUND} \\
\\
\frac{\text{dir}(lk) = d' \quad \text{lk_result}(lk, t_{\text{look}}, gm, \text{idx}(i)) = \text{None} \quad ([r], i, gm, d') \Downarrow t_{\text{look}}}{((?lk \ r) :: l, i, gm, d) \Downarrow \text{LKMismatch } lk \ t_{\text{look}}} \text{LOOKAROUNDFAIL}
\end{array}$$

Only 21 inductive rules to support all the control-flow! And we can reason by induction.

Backtracking Tree for $ab|(c)|a$ on string "ab"

Backtracking Tree for $ab|(c)|a$ on string "ab"**Faithfulness**

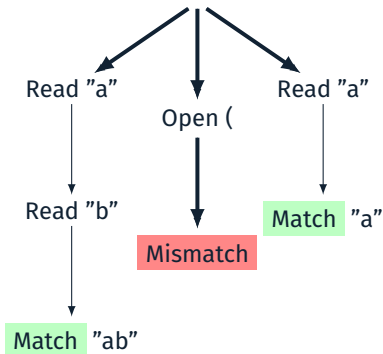
We proved that the leftmost Match node of the tree is equal to the result of Warblre.

Backtracking Tree for $ab|(c)|a$ on string "ab"**Faithfulness**

We proved that the leftmost Match node of the tree is equal to the result of Warblre.

More behaviors

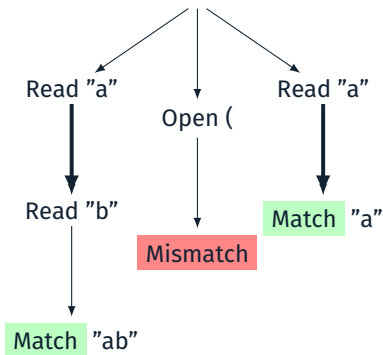
- Represents all paths explored by linear algorithms.
- Useful to reason about subregex equivalence.

Backtracking Tree for $ab|(c)|a$ on string "ab"**Faithfulness**

We proved that the leftmost **Match** node of the tree is equal to the result of Warblre.

More behaviors

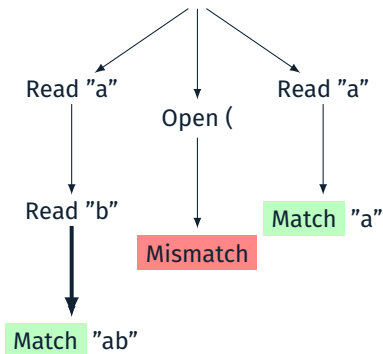
- Represents all paths explored by linear algorithms.
- Useful to reason about subregex equivalence.

Backtracking Tree for $ab|(c)|a$ on string "ab"**Faithfulness**

We proved that the leftmost **Match** node of the tree is equal to the result of Warblre.

More behaviors

- Represents all paths explored by linear algorithms.
- Useful to reason about subregex equivalence.

Backtracking Tree for $ab|(c)|a$ on string "ab"**Faithfulness**

We proved that the leftmost **Match** node of the tree is equal to the result of Warblre.

More behaviors

- Represents all paths explored by linear algorithms.
- Useful to reason about subregex equivalence.

Matching never fails

Every assertion holds, every array access is in bounds...

Theorem no_failure:

(* For all valid regex r and string s *)

$\forall r\ m\ s, \text{compileSubPattern } r = \text{Success } m \rightarrow$
 $\text{earlyErrors } r = \text{OK} \rightarrow$

(* the matcher cannot fail. *)

$m(\text{init_state } s) \neq \text{Failure}.$

Matching never fails

Every assertion holds, every array access is in bounds...

Theorem no_failure:

(* For all valid regex r and string s *)

$\forall r\ m\ s, \text{compileSubPattern } r = \text{Success } m \rightarrow$
 $\text{earlyErrors } r = \text{OK} \rightarrow$

(* the matcher cannot fail. *)

$m(\text{init_state } s) \neq \text{Failure}.$

Matching always terminates

For each quantifier, there is a maximum number of iterations:

Theorem termination:

(* For all valid regex r and string s *)

$\forall r\ m\ s, \text{compileSubPattern } r = \text{Success } m \rightarrow$
 $\text{earlyErrors } r = \text{OK} \rightarrow$

(* the matcher terminates. *)

$\text{terminates } (m(\text{init_state } s)).$

Matching never fails

Every assertion holds, every array access is in bounds...

Theorem no_failure:

(* For all valid regex r and string s *)

$\forall r m s, \text{compileSubPattern } r = \text{Success } m \rightarrow$
 $\text{earlyErrors } r = \text{OK} \rightarrow$

(* the matcher cannot fail. *)

$m(\text{init_state } s) \neq \text{Failure}.$

Matching always terminates

For each quantifier, there is a maximum number of iterations:

Theorem termination:

(* For all valid regex r and string s *)

$\forall r m s, \text{compileSubPattern } r = \text{Success } m \rightarrow$
 $\text{earlyErrors } r = \text{OK} \rightarrow$

(* the matcher terminates. *)

$\text{terminates } (m(\text{init_state } s)).$

Complexity Class

- Deciding whether a regex matches a string is PSPACE-complete.
- Without lookarounds, finding the top-priority match is OTP-complete.

Regex optimizers

Some optimizers, like the regex manipulation library `regexp-tree`, rewrite parts of a regex. `regexp-tree` replaces `r{min1,max1}r{min2,max2}` with `r{min1+min2,max1+max2}`.

Regex optimizers

Some optimizers, like the regex manipulation library `regexp-tree`, rewrite parts of a regex. `regexp-tree` replaces `r{min1,max1}r{min2,max2}` with `r{min1+min2,max1+max2}`.

We found counter-examples!

Regex optimizers

Some optimizers, like the regex manipulation library `regexp-tree`, rewrite parts of a regex. `regexp-tree` replaces $r\{\min1, \max1\}r\{\min2, \max2\}$ with $r\{\min1+\min2, \max1+\max2\}$.

We found counter-examples!

We proved that under some conditions, it can be correct. And it's different inside a lookbehind!

1 st quantifier \ 2 nd quantifier	$r\{\min2, \min2\}$	$r\{0, \max2\}$	$r\{0, \max2\}?$
$r\{\min1, \min1\}$	$\leftrightarrow \checkmark$	$\rightarrow \checkmark \quad \leftarrow \times$	$\rightarrow \checkmark \quad \leftarrow \times$
$r\{0, \max1\}$	$\leftarrow \checkmark \quad \rightarrow \times$	$\leftrightarrow \checkmark$	n/a
$r\{0, \max1\}?$	$\leftarrow \checkmark \quad \rightarrow \times$	n/a	$\leftrightarrow \times$

Regex optimizers

Some optimizers, like the regex manipulation library `regexp-tree`, rewrite parts of a regex. `regexp-tree` replaces $r\{\min1, \max1\}r\{\min2, \max2\}$ with $r\{\min1+\min2, \max1+\max2\}$.

We found counter-examples!

We proved that under some conditions, it can be correct. And it's different inside a lookbehind!

1 st quantifier \ 2 nd quantifier			
	$r\{\min2, \min2\}$	$r\{0, \max2\}$	$r\{0, \max2\}?$
$r\{\min1, \min1\}$	$\leftrightarrow^{\checkmark}$	$\rightarrow^{\checkmark} \leftarrow^{\times}$	$\rightarrow^{\checkmark} \leftarrow^{\times}$
$r\{0, \max1\}$	$\leftarrow^{\checkmark} \rightarrow^{\times}$	$\leftrightarrow^{\checkmark}$	n/a
$r\{0, \max1\}?$	$\leftarrow^{\checkmark} \rightarrow^{\times}$	n/a	$\leftrightarrow^{\times}$

Other uses

- **Fearless refactoring:** prove a new version of the spec is equivalent to the old one.
- **Feature encodings:** proved that anchors can be expressed with lookarounds. Next: atomic groups?
- **Encode many languages:** we're working on extending our semantics to support other regex flavors.

Algorithms you can trust

We verified the PikeVM linear algorithm, used in V8's *Experimental* engine!
With the fix to support JavaScript quantifiers.

Theorem `pike_vm_correctness`:

```
(* For any regex and string *)  
∀ (r:regex)(s:string),  
(* The result of the PikeVM is equal to Warblre's *)  
pike_vm(NFA r)(init_state s) = (Warblre.compile r) s.
```

Theorem `pike_vm_complexity`:

```
(* For any regex and string *)  
∀ (r:regex)(s:string),  
(* We reach a final state in a linear amount of steps *)  
reach_final(pike_vm(NFA r))(10 * size r * size s).
```

Verify the implementation, not just the algorithm!

Generate C/Wasm code for the PikeVM engine, with a proof between ECMAScript and C/Wasm semantics.

Verify the implementation, not just the algorithm!

Generate C/Wasm code for the PikeVM engine, with a proof between ECMAScript and C/Wasm semantics.

Linearity is not speed

On average, a backtracking implementation performs much better than the PikeVM.
Other linear engines (Rust Regex, RE2, Hyperscan, RE#...) achieve both linearity and efficiency.

Verify the implementation, not just the algorithm!

Generate C/Wasm code for the PikeVM engine, with a proof between ECMAScript and C/Wasm semantics.

Linearity is not speed

On average, a backtracking implementation performs much better than the PikeVM.
Other linear engines (Rust Regex, RE2, Hyperscan, RE#...) achieve both linearity and efficiency.

- Let's verify common linear engine optimizations: prefix acceleration, vectorization.

Verify the implementation, not just the algorithm!

Generate C/Wasm code for the PikeVM engine, with a proof between ECMAScript and C/Wasm semantics.

Linearity is not speed

On average, a backtracking implementation performs much better than the PikeVM.
Other linear engines (Rust Regex, RE2, Hyperscan, RE#...) achieve both linearity and efficiency.

- Let's verify common linear engine optimizations: prefix acceleration, vectorization.
- Let's verify other linear algorithms: Lazy DFA, OnePass, memoized backtracking.

Verify the implementation, not just the algorithm!

Generate C/Wasm code for the PikeVM engine, with a proof between ECMAScript and C/Wasm semantics.

Linearity is not speed

On average, a backtracking implementation performs much better than the PikeVM.
Other linear engines (Rust Regex, RE2, Hyperscan, RE#...) achieve both linearity and efficiency.

- Let's verify common linear engine optimizations: prefix acceleration, vectorization.
- Let's verify other linear algorithms: Lazy DFA, OnePass, memoized backtracking.
- Let's explore new optimizations!
Native compilation for the PikeVM.

CONCLUSION

For most JavaScript regexes, you could use a formally verified, linear-time, efficient engine!
We want to implement and prove that engine.

Linear algorithms · PLDI 24 Paper

Linear Matching of JavaScript Regular Expressions.
<https://github.com/LindenRegex/RegElk>

Mechanized Semantics · ICFP 24 Paper

A Coq Mechanization of JavaScript
Regular Expression Semantics.
<https://github.com/LindenRegex/Warblre>

Formal Verification · POPL 26 Paper

Formal Verification for JavaScript Regular Expressions:
a Proven Semantics and its Applications.
<https://github.com/LindenRegex/Linden>

Summary

- ECMAScript is in an unique position for linear-time matching.
- We can support lookarounds in linear time.
- We have mechanized semantics for proofs: properties, equivalences, algorithms...
- We are developing a formally verified, linear-time engine.

Summary

- ECMAScript is in a unique position for linear-time matching.
- We can support lookarounds in linear time.
- We have mechanized semantics for proofs: properties, equivalences, algorithms...
- We are developing a formally verified, linear-time engine.

Discussion

- Can we consider complexity for new proposals?
- Should we provide user control for linearity (l flag) or should it be heuristics-based?
- What properties/algorithms would you like to see verified?
- What were the reasons behind the unique semantic choices?